Drogaria Bem-Estar — Plataforma de Dados

Trabalho Final · Cenário 3 (rede de farmácias) · Luis Cesar Dantas Caricchio Costa, IBMEC Botafogo · 08/11/2026 ·
Dados reais até 30/09/2026; lote de 01–14/10/2026 **simulado** (ver §6)

1. Problema, produto e arquitetura

A rede de 20 lojas perde venda por **ruptura** e dinheiro por **produto vencido**, e compra de distribuidores que prometem prazos que não cumprem. O produto de dados é o **relatório diário de reposição** (HTML + CSV, publicado no Lake): o que repor com o prazo *real* de cada distribuidor, o que transferir entre lojas da mesma cidade, os lotes que vencem antes de vender, a pontualidade dos distribuidores e a perda por vencimento. Responde às 5 perguntas da diretoria com SQL só sobre a Gold (`sql/perguntas_negocio.sql`).

SQLite	**CSV semanal**	**JSON**	**JSONL**	**API**
vendas 36.357 · itens 50.350 · clientes 4.163 · produtos 73 · lojas 20	inventário loja×lote, 18 semanas, 34.895 linhas	fornecedores (9 registros, 8 ids)	eventos PDV, 122 dias, 61.212	pedidos de reposição, 7.691 (`sequencia`)

5 ELs incrementais → Landing → **Lake S3** SeaweedFS `bemestar-lake` → **Bronze** Parquet, texto + linhagem → **Silver** regras, HMAC quarentena → **Gate** % quarentena exit 1 = para → **Gold** DuckDB fatos + marts → **Produto** relatório + listas CSV

Airflow (`bemestar_pipeline`, 06:00, `max_active_runs=1`): sensores (`aguardar_api`; `aguardar_lake` espera se o Lake está fora, falha na hora com 403) → 5 ELs em paralelo → Landing no Lake → Bronze → Silver → gate (`max_quarentena_pct`=3) → Gold → validar → publicação (`--espelhar` nas camadas derivadas) + produto. Testado: execução completa verde, API derrubada (o sensor insiste), credencial errada (falha em 2 s), gate a 0,01% (para; o hash do `bemestar.duckdb` não mudou).

Incrementalidade. SQLite: marca d'água em `datetime(atualizado_em)` por tabela; itens seguem a venda-mãe. CSV/JSONL/JSON: SHA-256 de cada arquivo (nova versão do JSON é guardada, a mais recente vale). API: `after_sequence` guardado em estado, uma linha por versão de pedido. Cada EL tem seu arquivo de estado e grava de forma atômica. Silver mantém a **versão mais recente por chave**; Bronze guarda todas.

2. Qualidade: regras, quarentena e gate

Cada regra tem **tipo** (validade, completude, unicidade, integridade, consistência) e **ação**: *quarentena* (linha não confiável, vai com o motivo), *corrigir* (correção determinística e documentada) ou *alerta* (segue, mas é contada). Números do estado real (30/09), de `data/qualidade/relatorio_qualidade.json`:

Conjunto · regra	Tipo	Ação	Linhas
inventário · `linha_duplicada` (mesma loja×produto×lote×data)	unicidade	quarentena	178
inventário · `quantidade_negativa`	validade	quarentena	104
inventário · `produto_inexistente` (id 9999) / `lote_incompativel_com_produto`	integridade / consistência	quarentena	66 / 66
inventário · validade `MM/AAAA` trocada pela data exata do lote na API / último dia do mês	consistência	corrigir	4.299 / 825
inventário · `lote_vencido_na_contagem` (fato de negócio, base da perda)	consistência	alerta	32
eventos · `evento_duplicado` (reenvio do PDV, fica a 1ª) / `data_invalida` / `tipo_ausente` / `venda_inexistente`	unicidade, validade, completude, integridade	quarentena	519 / 97 / 84 / 57
eventos · horário em UTC convertido para Brasília / evento em arquivo de outro dia	consistência	corrigir / alerta	115 / 916
pedidos · `entregue_antes_de_solicitado` / `quantidade_invalida` / `produto_inexistente` / `status_incoerente`	consistência, validade, integridade	quarentena	27 / 18 / 16 / 6
pedidos · status `ENTREGUE`, `entregue `, `em transito` padronizados	consistência	corrigir	13
itens de venda · `produto_inexistente` / `lote_incompativel_com_produto` (mesmos 45 itens)	integridade / consistência	quarentena	45 / 45
itens de venda · `lote_ausente` (entra na venda, não baixa lote)	completude	alerta	105
vendas · `valor_total` negativo com itens positivos (|valor| = soma dos itens, conferido)	validade	corrigir	67

fornecedores · id repetido divergente / prazo "2 dias" / pedido mínimo negativo / CNPJ ausente	unicidade, consistência, validade, completude	quarentena / corrigir / corrigir / alerta	1 / 1 / 1 / 1
clientes · CPF só dígitos (aceito) / CPF inválido / e-mail inválido / telefone placeholder	consistência, validade	corrigir / alerta	30 / 10 / 19 / 40

Gate. Percentual de quarentena por conjunto no estado real: eventos 1,24% · inventário 0,99% · pedidos 0,87% · itens 0,09% · demais 0%; fornecedores 11,11% (1 de 9 linhas). **Limite padrão 3%** (parâmetro do DAG): os defeitos reais se concentram em ~1% e o limite dá ~2–3× de folga; um salto acima de 3% indica quebra da fonte (formato novo, id órfão em massa), não ruído. Conjuntos minúsculos usam **limite em linhas** (fornecedores e lojas: 2), pois 1 linha ruim de 9 seria 11%. O gate também confere que Silver+quarentena+versões substituídas = linhas da Bronze (contagem independente), então nada some sem explicação. Se bloqueia, sai com código 1 e a Gold anterior permanece.

3. Gold: grão de cada tabela

Tabela	Grão (uma linha por…)	Linhas	Por que esse grão
`fato_venda_item`	item de venda (`item_id`)	50.305	menor grão com loja, produto, lote e data: dá taxa de venda por loja×produto e baixa por lote (FEFO)
`fato_estoque_lote`	loja × produto × lote × data de contagem	34.549	a contagem semanal é a âncora do estoque; o lote carrega a validade
`fato_reposicao`	pedido (versão vigente)	7.624	mede prazo prometido × real, atraso e pedidos em trânsito
`fato_devolucao`	evento de devolução	170	ajusta o estoque; o evento é o fato
dim_loja · dim_produto · dim_fornecedor · dim_data	chave de negócio (data: 425 dias)	20 · 73 · 8	chaves substitutas; `nome` só aqui
`mart_estoque_loja_produto`	loja × produto	1.377	dias de estoque, situação e quantidade sugerida (P1)
`mart_estoque_lote` / `mart_risco_vencimento`	loja × produto × lote	2.166	estoque estimado por lote; sobra prevista na validade em fila FEFO (P2)
`mart_perda_vencimento`	loja × produto × lote vencido	40	unidades e valor perdidos (P4); excludente com risco
`mart_pontualidade_fornecedor` · `mart_transferencia`	fornecedor · produto × origem × destino	8 · 27	P3 e P5
`mart_receita_retida`	loja × categoria × mês (k ≥ 5)	160	agregado, por LGPD

Estoque de hoje. Não existe contagem diária: estoque = última contagem do lote − vendas do lote desde então + entregas. Em 97% das semanas por lote a conta fecha exatamente; o ruído restante (~3%: quebra, furto, erro de contagem) é a incerteza declarada. `validar.py` roda 47 verificações (grão único, chaves sem órfãs, reconciliação com a Silver, invariantes, LGPD): 47/47. Ela já pegou falhas reais: R$ 2.644,02 de diferença que eram os itens em quarentena (a conciliação passou a ser *Gold + quarentena = total da venda*) e um lote contado em perda e em risco ao mesmo tempo.

4. LGPD: decisões do cenário

- **A reposição não precisa de pessoa.** A ligação cliente↔medicamento é cortada na **Silver**: `vendas` perde `cliente_id` (fica `cliente_identificado`) e a Gold nem lê `clientes`. Nenhuma tabela da Gold permite descobrir o medicamento de uma pessoa identificável (verificado em `validar.py` por nome de coluna).
- **Pseudonimização:** `cliente_id` e CPF viram HMAC-SHA256 com chave em `PSEUDO_CHAVE` / Variable `pseudo_secret` (nunca no código). SHA-256 puro seria revertido por lista de CPFs; HMAC sem a chave não.
- **Descartados na Silver:** nome, e-mail, telefone, nascimento, `operador_id` (funcionário) e `contato_email` do distribuidor. CNPJ (pessoa jurídica) fica na Silver, fora da Gold.
- `receita_retida` **(9.769 eventos) é dado de saúde:** só existe como `mart_receita_retida`, por loja × categoria × mês, com supressão de células com menos de 5 eventos (medido: 160 células, 0 abaixo de k).
- **Zonas restritas:** Landing e Bronze guardam o bruto (com CPF e nome) para auditoria. A quarentena é publicada com as colunas pessoais **mascaradas** na entrada. No laboratório há uma só credencial; em produção seriam prefixos com IAM, consumidores lendo só `warehouse/` e `relatorios/`.

5. Produto de dados: evidência de que funciona

Regra. Repor quando `(estoque vendável + em trânsito) / taxa 28 d < p90 do prazo real do distribuidor + 1 dia`; quantidade = taxa × (p90 + 14) − estoque − em trânsito, com o pedido mínimo do distribuidor indicado ao lado de cada sugestão. Só conta estoque de lotes não vencidos. Transferência só dentro da mesma cidade, de lojas em excesso (mais de 60 dias de cobertura), sem deixar o doador abaixo de 45 dias.

5.1 Margem de prazo: prazo real × prometido (estado de 30/09)

Só 72,3% dos 7.421 pedidos entregues chegaram no prazo. Nova Saúde Atacadista (34,9% no prazo, p90 = 9 dias) e Profarma Leste (35,6%, p90 = 9) prometem 2 dias; Santa Cruz promete 7 e tem p90 de 8; os outros cinco ficam entre 79,8% e 83,8%. **Regra ingênua** (cobertura menor que o prazo cadastrado): 16 combinações. **Regra do produto**: 37 (12 rupturas e 25 a repor), incluindo as 16. **21 casos (57%) só aparecem com o prazo real**; 14 dos 37 são desses dois distribuidores. Sugestão: 129 unidades, R$ 5.800,71 a preço de venda (o custo não existe nas fontes).

5.2 Respostas à diretoria (30/09, dados reais)

Pergunta	Resultado
P1 dias de estoque	1.377 combinações loja×produto: 1.129 ok, 175 excesso (mediana 98 dias), 36 sem giro, 25 repor (mediana 4,7 dias), 12 ruptura
P2 lotes que vencem antes de vender	60 lotes, 939 unidades, R$ 46.601,71 em risco em 90 dias; 31 lotes (R$ 30.371,04) vencem em até 30 dias
P3 distribuidores fora do prazo	Nova Saúde e Profarma Leste (ver 5.1); atraso médio dos atrasados de 4,8 dias nesses dois
P4 perda por vencimento	40 lotes, 437 unidades, **R$ 23.936,63** (jul–set; 41% uso contínuo, 33% dermocosmético). Valor a preço de venda, só lotes vistos nas contagens: é piso, não total
P5 sobra × falta	27 transferências possíveis, 130 unidades, todas na mesma cidade

5.3 Antes × depois do lote (simulado)

Listas de 30/09 contra o que aconteceu em 01–14/10 (`sql/avaliacao_produto.sql`): (a) das 41 combinações apontadas, **61,0%** (25) estavam sem nenhuma unidade vendável na contagem de 12/10, contra 34,4% (449 de 1.305) das não apontadas; (b) dos 7 lotes apontados que venceriam entre 06 e 14/10, 6 (85,7%) ainda tinham unidades na última contagem antes de vencer; (c) as 9 perdas por vencimento de outubro (todas) já estavam na lista de risco. Limite: o lote foi gerado por mim e as vendas não dependem das recomendações; o teste mostra que a lista aponta para onde o estoque realmente acaba, não que o produto reduza perdas. Com 7 lotes a amostra de (b) é pequena.

6. Lote incremental: o que foi provado

> O `lote_incremental/` do professor não veio no material. `tools/simular_lote.py` aplica um lote equivalente (semente fixa, idempotente): 30 clientes alterados + 40 novos, 12 reajustes de preço + 3 produtos novos, 3.394 vendas novas (e 5 cancelamentos de vendas antigas), 150 pedidos em trânsito → entregue, 763 pedidos novos, contagens de 05/10 e 12/10 e nova versão de `fornecedores.json`. Todo número de outubro é **simulado**.

Camada	Antes	Depois	Leitura
Bronze · vendas / itens / produtos	36.357 / 50.350 / 73	39.756 / 55.581 / 88	Bronze acumula versões (produtos 88 = 73 + 15 alterados/novos)
Silver · produtos / pedidos (1 por pedido)	73 / 7.624	76 / 8.388	Silver tem uma linha por chave: reajuste substitui, não duplica
Gold · fato_venda_item / dim_produto	50.305 / 73	55.524 / 76	grão único (validar 47/47)
Pedidos em trânsito / entregues	203 / 7.421	434 / 7.954	150 atualizados para entregue

Os ELs capturaram só o novo: SQLite 15 produtos, 70 clientes, 3.399 vendas, 5.233 itens; 40 CSVs; 14 JSONL; 1 nova versão do JSON; API 913 registros (sequência 7.691 → 8.604). **Segunda execução sem dado novo: 0 capturas, 549 arquivos pulados, 0 linhas novas na Bronze e `contagens.sql` idêntico** (`diff` vazio).

7. Dificuldades: problema, causa, solução

- **Silver quebrou com `invalid escape sequence \1`.** O DuckDB usa RE2, sem retrovisor; a regra de telefone-placeholder (21999999999) usava `(\d)\1+`. Troquei por comparação com `replace()` do primeiro dígito.
- **A conciliação da Gold acusou R$ 2.644,02 de diferença.** Eram os itens em quarentena (produto 9999), que ainda compõem o total da venda. A regra virou Gold + quarentena = total.
- **Lote vencendo "hoje" contado duas vezes** (em perda e em risco). Perda passou a ser `validade < data_ref`; e, após o lote, o `validar` pegou lote já vencido aparecendo em risco: risco agora só considera lotes

não vencidos, e eles saem da fila FEFO.
- **O gate funcionou contra mim:** a 1ª versão do simulador gerou códigos de lote fora do padrão `Lppp-nnnn` e fornecedor errado em produtos novos, o que mandou 9,81% dos pedidos para a quarentena e o gate bloqueou a Gold. Corrigi o simulador, não a regra.
- **Linha de base contaminada:** ao reconstruir o "antes" a API ainda servia o lote; só comparei as contagens depois de subir uma API sobre as fontes originais (contagens idênticas às de antes do lote).
- **Sem custo de compra e sem contagem diária.** Valores a preço de venda; estoque estimado com ruído de ~3%. Ambos declarados no produto.

Fontes dos números: `data/qualidade/relatorio_qualidade.json` e `gate.json` (cópias do estado real em `docs/*_30set.*`), `docs/contagens_antes|depois_do_lote.txt`, `docs/respostas_diretoria_30set.txt`, `docs/avaliacao_produto.txt`, `data/qualidade/validacao_gold.json`.